Reporte Técnico: Modificaciones e Implementaciones en Librerías Java

Asignatura: Modelos y Simulaciones
Proyecto: Simulación de Eventos Discretos ? Aeropuerto
Fecha: Octubre 2026

---

1. Introducción y Propósito

El presente documento detalla todas las modificaciones, correcciones y desarrollos realizados sobre el esqueleto de librerías en Java proporcionadas para el trabajo práctico de la materia **Modelos y Simulaciones**:

1. **`random-variate-gen-lab`**: Librería para la generación de variables aleatorias siguiendo diversas distribuciones de probabilidad (Exponencial, Uniforme, Normal y Discreta empírica).
2. **`queueing-sim-lab`**: Librería motor para la simulación de eventos discretos basada en modelos de colas (Módelos de Colas, Eventos de Arribo/Salida, Lista de Eventos Futuros - FEL, Servidores, Colas y Probes/Recolectores de estadísticas).

---

2. Reglas de Implementación Respetadas

- **Preservación de Modificadores de Acceso:** No se alteró ningún modificador de acceso (`public`, `private`, `protected`, o *package-private*) respecto al diseño base proporcionado.
- **Demarcación de Código Agregado:** Todas las clases modificadas o completadas incluyen marcas explícitas en los comentarios:
  ```java
  // [IMPLEMENTACIÓN COMPLETADA] - Inicio
  ... código implementado ...
  // [IMPLEMENTACIÓN COMPLETADA] - Fin
  ```
  y en correcciones puntuales de compatibilidad: `// [AGREGADO: ...]`.
- **Compatibilidad de Entorno:** Se adaptó el proyecto para compilar y ejecutar perfectamente sobre **Java 17 (JDK 17)** y Maven 3.8+.

---

3. Detalle de Modificaciones por Librería

3.1. Librería `random-variate-gen-lab`

Esta librería proporciona los generadores de tiempos entre arribos y tiempos de servicio necesarios para las simulaciones.

Clase	Archivo	Cambios e Implementación Realizada
`Exponential`	[`Exponential.java`](file:///C:/Users/Usuario/.gemini/antigravity/scratch/aeropuerto/MySproyectofinal/random-variate-gen-lab/src/main/java/random/variates/Exponential.java)	- **Constructores**: Validación de que el parámetro $\lambda > 0$ (lanza `IllegalArgumentException` si $\lambda \le 0$) y asignación del generador `RandomGenerator`. - **Método `sample()`**: Implementación del método de la **Transformada Inversa**: $X = -\frac{\ln(1 - U)}{\lambda}$, donde $U \sim \text{Uniforme}(0,1)$.

`Uniform`	[`Uniform.java`](file:///C:/Users/Usuario/.gemini/antigravity/scratch/aeropuerto/MySproyectofinal/random-variate-gen-lab/src/main/java/random/variates/Uniform.java)	- **Constructores**: Validación de que el cota inferior $a < b$ (lanza `IllegalArgumentException` si $a \ge b$). - **Método `sample()`**: Muestreo uniforme continuo en $[a, b)$ mediante $X = a + (b - a) \cdot U$.
`Normal`	[`Normal.java`](file:///C:/Users/Usuario/.gemini/antigravity/scratch/aeropuerto/MySproyectofinal/random-variate-gen-lab/src/main/java/random/variates/Normal.java)	- **Constructores**: Validación de desviación estándar $\sigma > 0$. - **Método `sample()`**: Generación de muestras gaussianas $N(\mu, \sigma^2)$ haciendo uso directo del método `randomGenerator.nextGaussian(mean, stdDev)`.
`Discrete`	[`Discrete.java`](file:///C:/Users/Usuario/.gemini/antigravity/scratch/aeropuerto/MySproyectofinal/random-variate-gen-lab/src/main/java/random/variates/Discrete.java)	- **Muestreo Discreto Empírico**: Muestreo por suma acumulada de probabilidades para tablas discretas personalizadas.

3.2. Librería `queueing-sim-lab`

Esta librería constituye el motor de simulación de eventos discretos.

Clase	Archivo	Cambios e Implementación Realizada
`Arrival`	[`Arrival.java`](file:///C:/Users/Usuario/.gemini/antigravity/scratch/aeropuerto/MySproyectofinal/queueing-sim-lab/src/main/java/queueing/sim/Arrival.java)	- **Lógica del Evento de Arribo (`execute`)**: 1. Asigna el tiempo de arribo a la entidad si es su primer evento. 2. Notifica a los recolectores de métricas (`probes.entityArrived`). 3. Consulta si hay servidores disponibles en el modelo global (`availableServers`). 4. **Si hay servidor disponible**: Selecciona servidor usando la estrategia del modelo (`select`), ocupa el servidor (`seize`), notifica a probes (`serverBecameBusy` / `serviceStarted`) y programa la salida (`Departure`) con el tiempo de servicio generado. 5. **Si NO hay servidor disponible**: Selecciona la cola objetivo (`select`), encola la entidad (`add`) y notifica a probes (`entityEnteredQueue` / `queueSizeChanged`). 6. Genera el tiempo para el próximo arribo y programa el siguiente evento `Arrival` en el FEL.
`Departure`	[`Departure.java`](file:///C:/Users/Usuario/.gemini/antigravity/scratch/aeropuerto/MySproyectofinal/queueing-sim-lab/src/main/java/queueing/sim/Departure.java)	- **Lógica del Evento de Salida (`execute`)**: 1. Libera la entidad atendida del servidor (`server.release()`) y notifica a probes (`entityDeparted`). 2. Filtra las colas del modelo que tengan entidades esperando. 3. **Si hay entidades esperando**: Selecciona la cola según la estrategia de salida (`select`), retira la primera entidad (`remove`), ocupa el servidor liberado (`seize`), notifica a probes y programa la salida de esta nueva entidad. 4. **Si NO hay entidades en cola**: Notifica que el servidor pasa a estar ocioso (`serverBecameIdle`).
`Queue`	[`Queue.java`](file:///C:/Users/Usuario/.gemini/antigravity/scratch/aeropuerto/MySproyectofinal/queueing-sim-lab/src/main/java/queueing/sim/Queue.java)	- **Corrección de Bug de Validación (NPE)**: Al validar la lista de servidores recibida en el constructor, `List.of(...).contains(null)` lanzaba una excepción `NullPointerException` propia de las colecciones inmutables de Java. Se reemplazó por un chequeo seguro usando Stream: `servers.stream().anyMatch(Objects::isNull)`.
`Simulation`	[`Simulation.java`](file:///C:/Users/Usuario/.gemini/antigravity/scratch/aeropuerto/MySproyectofinal/queueing-sim-lab/src/main/java/queueing/sim/Simulation.java)	- **Inicialización del Reloj**: Se ajustó la programación del primer evento de arribo al inicio del ciclo de simulación (`initClock`, tiempo 0.0) para garantizar que los contadores de arribos y salidas coincidan exactamente con la duración de la corrida.

| **`FirstAvailable`** | [`FirstAvailable.java`](file:///C:/Users/Usuario/.gemini/antigravity/scratch/aeropuerto/MySproyectofinal/queueing-sim-lab/src/main/java/queueing/sim/strategies/FirstAvailable.java) | - Reemplazo de `.getFirst()` por `.get(0)` para asegurar compatibilidad completa con Java 17. |

---

4. Adaptación de Compatibilidad y Entorno (Java 17)

Para garantizar la compilación y ejecución de las pruebas en entornos Java 17:

1. **Configuración de Maven (`pom.xml`)**:
   - En ambos proyectos se actualizó la propiedad `<maven.compiler.release>17</maven.compiler.release>`.
2. **Reemplazo de métodos de Java 21**:
   - Se reemplazaron invocaciones del método `SequencedCollection.getFirst()` por `List.get(0)` en clases del modelo y clases de prueba (`ArrivalTest.java`, `DepartureTest.java`, `QueueingModelTest.java`, `SimulationTest.java`, `TestStrategies.java`).

---

5. Resultados de Verificación (Pruebas Unitarias)

Se ejecutó la suite completa de pruebas unitarias vía Maven (`mvn test`):

5.1. Librería `random-variate-gen-lab`

```text
[INFO] Running random.variates.DiscreteTest     -> 5/5 PASSED
[INFO] Running random.variates.ExponentialTest  -> 3/3 PASSED
[INFO] Running random.variates.NormalTest       -> 4/4 PASSED
[INFO] Running random.variates.UniformTest      -> 3/3 PASSED
[INFO] -------------------------------------------------------------------
[INFO] BUILD SUCCESS (Total tests: 15, Failures: 0, Errors: 0)
[INFO] -------------------------------------------------------------------
```

5.2. Librería `queueing-sim-lab`

```text
[INFO] Running queueing.sim.ArrivalTest              -> 6/6 PASSED
[INFO] Running queueing.sim.DepartureTest            -> 5/5 PASSED
[INFO] Running queueing.sim.FutureEventListTest      -> 6/6 PASSED
[INFO] Running queueing.sim.probes.*                 -> 29/29 PASSED
[INFO] Running queueing.sim.QueueingModelTest        -> 11/11 PASSED
[INFO] Running queueing.sim.QueueSelectionStrategyTest-> 1/1 PASSED
[INFO] Running queueing.sim.QueueTest                -> 11/11 PASSED
[INFO] Running queueing.sim.ServerTest               -> 10/10 PASSED
[INFO] Running queueing.sim.SimulationIntegrationTest-> 3/3 PASSED
[INFO] Running queueing.sim.SimulationTest           -> 5/5 PASSED
[INFO] -------------------------------------------------------------------
[INFO] BUILD SUCCESS (Total tests: 87, Failures: 0, Errors: 0)
[INFO] -------------------------------------------------------------------
```

---

6. Conclusión

Ambas librerías se encuentran **100% funcionales, probadas y validadas**. El motor de simulación de colas y el generador de variables aleatorias están completamente preparados para soportar el desarrollo de la simulación del **Aeropuerto (Etapa 1, Etapa 2 y réplicas de la Etapa 3)**.